Домашнее задание по лекции «Стандарты форматирования сообщений

Задание 1. Выбор веб-приложения и определение форматов

Приложение: Telegram (мессенджер).

Используемые форматы:

- **MTProto** — собственный бинарный протокол Telegram для обмена между клиентом и сервером (основан на бинарной сериализации).
- **JSON** — используется в Bot API и в некоторых публичных REST-эндпоинтах.
- **Protocol Buffers / собственные схемы** — во внутренних сервисах (частично).

Обоснование: Telegram — высоконагруженная система с миллионами одновременных соединений. Публичная документация MTProto и Bot API подтверждает использование бинарного протокола для клиент-серверного обмена.

Задание 2. Обоснование выбора формата и альтернативы

Почему выбран MTProto (бинарный формат):

- Минимальный размер сообщений → экономия трафика (особенно важно для мобильных сетей).
- Высокая скорость сериализации/десериализации → низкая задержка.
- Строгая схема → контроль типов и предсказуемость.
- Поддержка шифрования на уровне протокола.

Альтернативы и почему не подходят:

Альтернатива	Почему не подходит
JSON	Избыточен, медленный парсинг, большой размер — критично при миллионах сообщений в секунду
XML	Ещё более избыточен и медленный, устаревший
Protobuf	Хорош, но не даёт встроенного шифрования и привязки к транспорту; Telegram нужен единый протокол с уровнями шифрования
MessagePack	Компактнее JSON, но не поддерживает схемную эволюцию и не обеспечивает нужный уровень контроля

Задание 3. Пример данных в трёх форматах

Данные: профиль пользователя.

3.1. JSON

```
json
{
  "user_id": 12345,
  "name": "Иван Петров",
  "email": "ivan@example.com",
  "age": 30,
  "is_active": true
}
```

Размер: ≈ 110 байт.

3.2. XML

```
xml
<user>
  <user_id>12345</user_id>
  <name>Иван Петров</name>
  <email>ivan@example.com</email>
  <age>30</age>
  <is_active>true</is_active>
</user>
```

Размер: ≈ 170 байт.

3.3. Protocol Buffers (схема .proto)

```
proto
message User {
  int32 user_id = 1;
  string name = 2;
  string email = 3;
  int32 age = 4;
  bool is_active = 5;
}
```

Размер бинарного сообщения: ≈ 40–50 байт.

Сравнение

Формат	Размер	Читаемость
JSON	~110 Б	Высокая
XML	~170 Б	Средняя (много «шума»)
Protobuf	~45 Б	Нулевая без схемы

Вывод: Protobuf компактнее JSON примерно в 2,5 раза и XML — почти в 4 раза, но полностью нечитаем для человека.

Задание 4. Переход с JSON на Protobuf для Telegram

Преимущества:

- Уменьшение размера сообщений в 2–3 раза → экономия трафика и нагрузки на сеть.
- Ускорение сериализации/десериализации → снижение задержки.
- Строгая типизация → меньше ошибок на этапе разработки.
- Схемная эволюция → можно добавлять поля без ломания совместимости.
- Генерация кода для множества языков → упрощение поддержки клиентов.

Ограничения:

- Нечитаемость → сложнее отладка.
- Нужно поддерживать схему в актуальном состоянии.
- Требуется Schema Registry или другой механизм версионирования.
- Для веб-клиентов (JavaScript) Protobuf менее удобен, чем JSON.
- Bot API, ориентированный на сторонних разработчиков, выигрывает от простоты JSON.

Вывод: для внутренних сервисов переход оправдан, для публичного Bot API — нет.

Задание 5. Система, для которой XML остаётся оптимальным выбором

Пример: государственная система документооборота или корпоративная интеграция с legacy-системами (например, банковский шлюз, взаимодействующий с СОАП-сервисами).

Обоснование:

- **Стандартизация:** XML поддерживается стандартами (XSD, XSLT, XPath), что критично для государственных и корпоративных регламентов.

- **Legacy-совместимость:** большинство старых корпоративных систем используют XML (SOAP, WSDL).
- **Сложные структуры:** XML удобен для описания документов с вложенными элементами, атрибутами, пространствами имён.
- **Валидация:** XSD позволяет строго проверять структуру документа.
- **Человекочитаемость:** важно для юридически значимых документов и ручной проверки.

Пример: система электронного документооборота «Диадок» или государственные порталы, обменивающиеся XML-документами по ГОСТ.